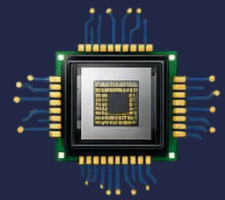
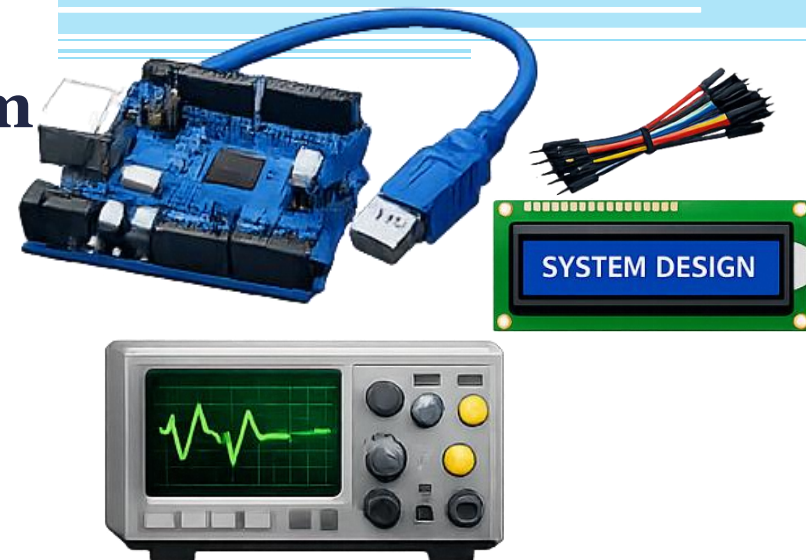


CS-301 Microprocessor- Based System Design



Lecture 1: Principles of Microprocessor based System Design and Embedded Programming

Course Teacher: Syeda Ramish Fatima



Course Outline

Principles of Microprocessor based Design and Embedded Programming

Programmable Peripheral Interfacing Circuitry

IO Pins, Pin Functions, Alternate Pin Functions

Interfacing microprocessors with basic IO devices (switches, LEDs, seven-segments),
Related Embedded Coding techniques

Interfacing microprocessors with basic IO devices (LCDs) LCD Controllers, Embedded
coding and design examples

Interfacing microprocessors with basic IO devices (keypads) Keypad Encoder IC,
Embedded Coding and design examples

Analog-to Digital Convertors (on chip and off chip)

Interfacing sensors with microprocessors); design examples

Timers and Counters

Oscillators and clock circuits

Pulse width Modulation (Design Examples)

Midterm Exam+ Some more design examples

Serial Communication protocols in microprocessor based systems

USART (protocol and design examples)

I²C (protocol and design examples)

On chip and off-chip memory interfacing techniques

Interrupts

Miscellaneous topics (Power modes, opto-couplers; interfacing with motors, IO bus
expansion)

Course Learning Outcomes (CLO's)

- [Course Profile_CS_301_MicroprocessorBasedSystemDesign.pdf](#)

Books

- [Atmel-2513-8-bit-AVR-Microcontroller-ATmega162_Datasheet](#)
- **Source:** <https://www.microchip.com/>
- **The AVR microcontroller and Embedded systems –Using Assembly and C**
by Muhammad Ali Mazidi, sarmad Naimi, and Sepehr Naimi

Sessional Criteria

Assessment Type	Marks	Schedule (Week No.)
Midterm Exam	14	9
Quizzes	6	6, 10
Assignment/Mini- Project/Group Activity	10	1-15
Total Sessional Marks	30	

Motivation

- **As Computer Engineering Students**
 - General Purpose Applications
 - Embedded Applications
- **As Future Innovators**
 - Internet-of-Things
 - AI accelerators
 - Digital system Design (FPGAs and Microprocessors)
 - Neuromorphic Computing
- **Levels of Abstraction**
 - Arduino, Raspberry Pi
 - Making your own boards
 - Using any microprocessors or microcontrollers

MBSD Vs DLD (Main Difference)

Aspect	Microprocessor-Based Design	Digital Logic Design (Off-the-Shelf ICs)
Core concept	Uses a programmable CPU to control the system	Uses fixed-function logic ICs (gates, counters, multiplexers, etc.)
Control method	Software (program stored in memory)	Hardware wiring (logic connections)

MBSD Vs DLD

(Flexibility & Complexity)

Feature	Microprocessor-Based	Digital Logic IC-Based
Flexibility	Very high (change behavior by changing software)	Very low (hardware must be rewired or redesigned)
Complexity handling	Best for complex tasks	Best for simple, fixed tasks
Design modification	Easy (reprogramming)	Difficult (hardware changes)

MBSD Vs DLD (Components Used)

Microprocessor-Based System

Microprocessor / Microcontroller

RAM & ROM

I/O interfaces

Clock and support circuitry

Digital Logic System

Logic gates (AND, OR, NOT)

Flip-flops

Counters, decoders, encoders

Multiplexers, registers

MBSD Vs DLD(Speed & Performance)

Aspect	Microprocessor-Based	Digital Logic IC-Based
Execution speed	Slower (instruction-by-instruction execution)	Very fast (parallel hardware operation)
Real-time response	Limited by software execution	Immediate hardware response

MBSD Vs DLD (Cost and Power)

Aspect	Microprocessor-Based	Digital Logic IC-Based
Initial cost	Higher (CPU + memory + peripherals)	Lower for simple designs
Power consumption	Generally higher	Lower for small logic systems

MBSD Vs DLD (Development time)

Aspect	Microprocessor-Based	Digital Logic IC-Based
Design time	Shorter for complex systems	Longer as complexity increases
Debugging	Software debugging tools available	Hardware debugging is harder

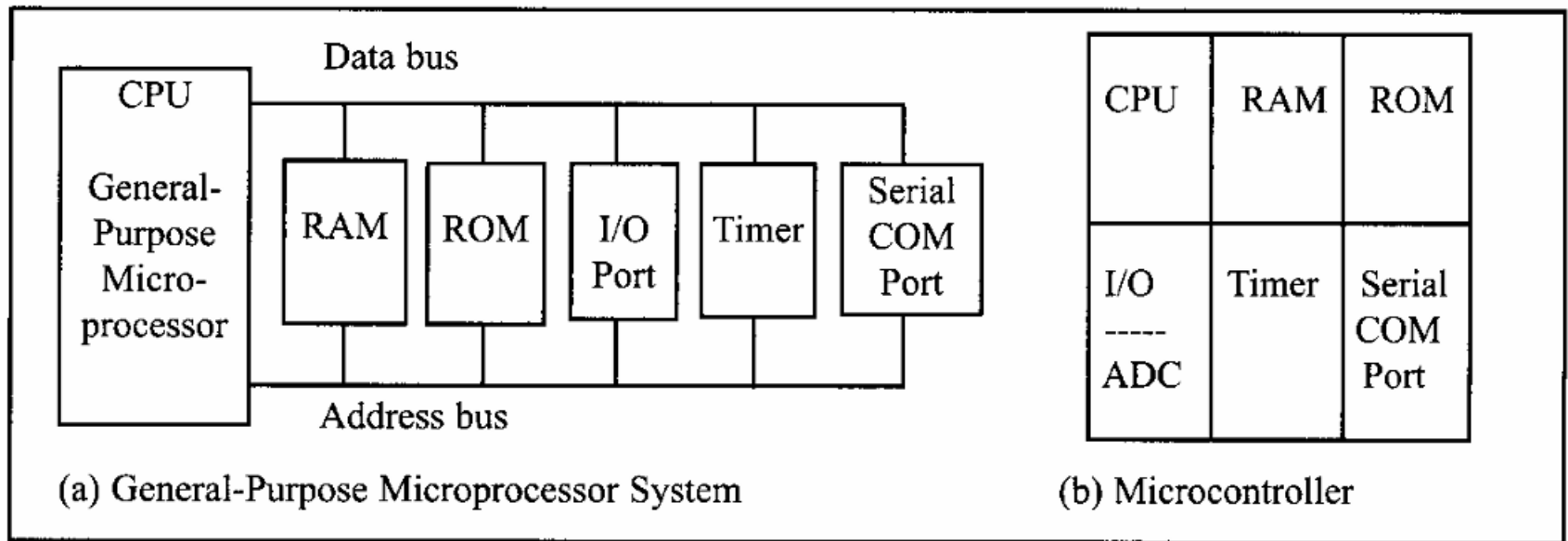
MBSD Vs DLD (Typical Applications)

- **Microprocessor-Based Design**
 - General-Purpose Computing
 - Washing machines
 - Traffic light controllers
 - Embedded systems
 - Industrial automation
 - IoT devices
- **Digital Logic Design (Off-the-Shelf ICs)**
 - Simple counters and timers
 - Alarm circuits
 - Basic control logic
 - Educational/training circuits



Can you develop a processor using techniques you learnt in DLD?

Components of a Microprocessor/Microcontroller-Based System



Components of a Microprocessor/Microcontroller-Based System

- **Microprocessor (MPU)**

- CPU-centric device; requires external memory and peripherals
- Optimized for high computational throughput
- Typically runs rich operating systems (Linux, Android)
- Higher power consumption and system cost
- Used in general-purpose computing and high-performance embedded systems

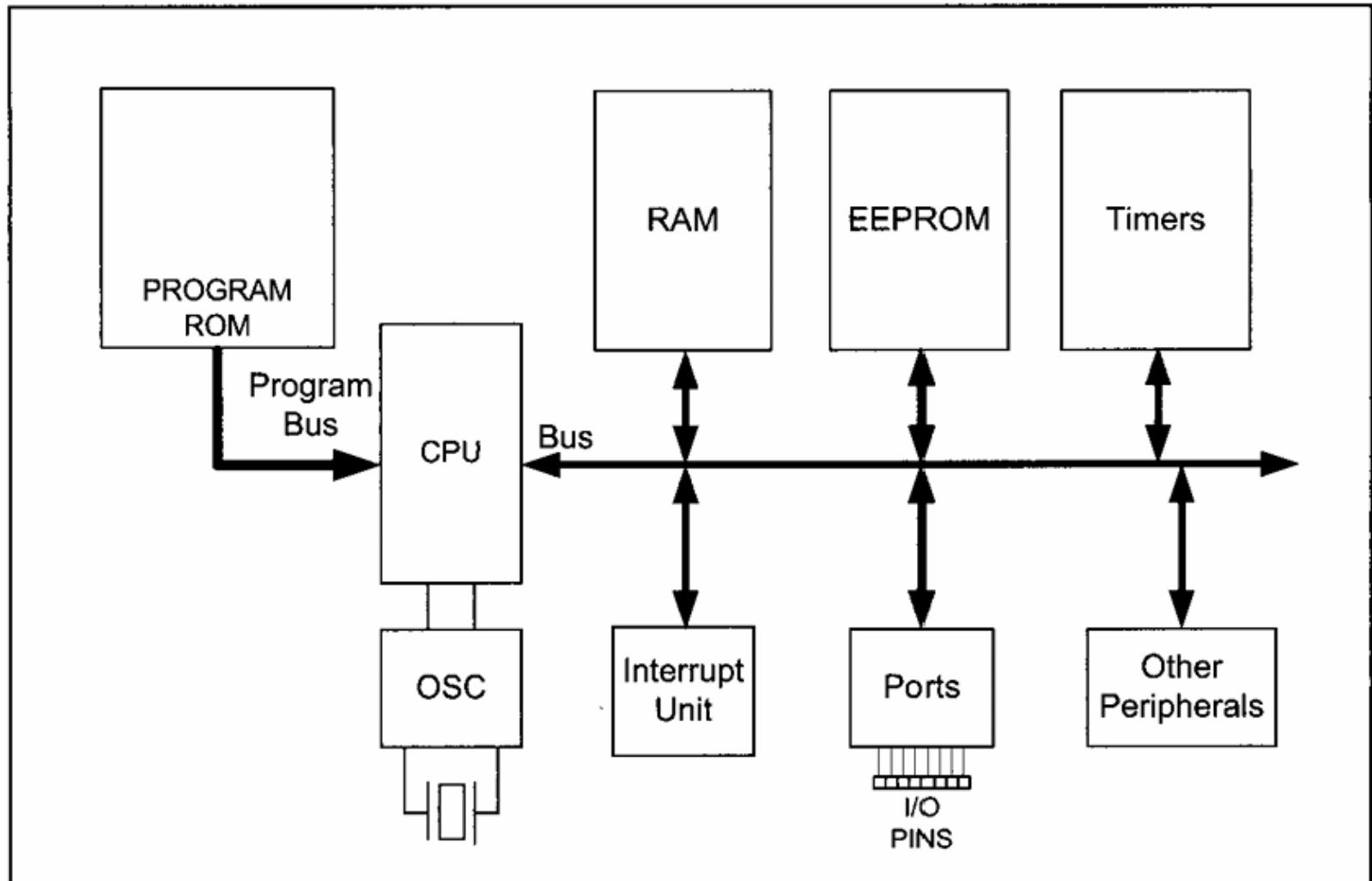
- **Microcontroller (MCU)**

- System-on-Chip (SoC): CPU + Flash + RAM + peripherals on one chip
- Optimized for deterministic, real-time control
- Executes bare-metal code or RTOS
- Low power consumption and compact hardware
- Used in control-oriented and resource-constrained embedded systems

Components of a Microprocessor/Microcontroller-Based System

- **Boundary Is Blurring (Modern Perspective)**
 - Modern microcontrollers integrate advanced features: DSP units, floating-point units, AI accelerators, high-speed connectivity
 - High-end MCUs can now run embedded Linux, traditionally associated with microprocessors
 - Many microprocessors integrate on-chip peripherals, memory controllers, and I/O, resembling MCUs
 - Both now follow SoC design philosophies, differing mainly in degree of integration and intended workload
 - Market trend favors application-driven classification rather than strict architectural definitions

Simplified View of AVR Microcontrollers



Selecting a Microcontroller

- The first and foremost criterion in choosing a microcontroller is that it must meet the task at hand efficiently and cost effectively.
 - In analyzing the needs of a microcontroller-based project, we must first see whether an 8-bit, 16-bit, or 32-bit microcontroller can best handle the computing needs of the task most effectively.
- Among other considerations in this category are:
 - Speed. What is the highest speed that the microcontroller supports?
 - Packaging. Does it come in a DIP (dual inline package) or a QFP (quad flat package), or some other packaging format? This is important in terms of space, assembling, and prototyping the end product.

Selecting a Microcontroller

- Power consumption. This is especially critical for battery-powered products.
- The amount of RAM and ROM on the chip.
- The number of I/O pins and the timer on the chip.
- Ease of upgrade to higher-performance or lower-power-consumption versions.
- Cost per unit. This is important in terms of the final cost of the product in which a microcontroller is used. For example, some microcontrollers cost 50 cents per unit when purchased 100,000 units at a time.

Selecting a Microcontroller

- The second criterion in choosing a microcontroller is how easy it is to develop products around it.
 - Key considerations include the availability of an assembler, a debugger, a code-efficient C language compiler, an emulator, technical support, and both in-house and outside expertise.
 - In many cases, third-party vendor (i.e., a supplier other than the chip manufacturer) support for the chip is as good as, if not better than, support from the chip manufacturer.

Selecting a Microcontroller

- The third criterion in choosing a microcontroller is its ready availability in needed quantities both now and in the future.
 - For some designers this is even more important than the first two criteria.
 - Currently, of the leading 8-bit micro controllers, the 8051 family has the largest number of diversified (multiple source) suppliers.
 - (Supplier means a producer besides the originator of the microcontroller.)
 - In the case of the 8051, which was originated by Intel, many companies also currently produce the 8051.

Selecting a Microcontroller

- Notice that Freescale Semiconductor (Motorola), Atmel, Zilog, and Microchip Technology have all dedicated massive resources to ensure wide and timely availability of their products because their products are stable, mature, and single sourced.
- In recent years, companies have begun to sell Field-Programmable Gate Array (FPGA) and Application-Specific Integrated Circuit (ASIC) libraries for the different applications.
- *Note: Atmel was merged with Microchip in the year 2016.*

HLL Programming VS Embedded Programming

- High-Level Language (HLL) Programming
 - Targeted at general-purpose computing platforms
 - Hardware details are abstracted by the OS and runtime
 - Execution is typically non-real-time
 - Emphasis on portability, productivity, and maintainability
 - Resources (memory, storage, CPU) are abundant and dynamically managed
 - Examples: Python, Java, C++ on PCs/servers

HLL Programming VS Embedded Programming

- **Embedded Programming**
 - Targeted at specific hardware platforms (MCUs/MPUs)
 - Direct interaction with hardware (registers, peripherals, interrupts)
 - Often real-time and time-deterministic
 - Emphasis on efficiency, reliability, and predictability
 - Resources are limited and statically managed
 - Examples: C/C++ for microcontrollers, bare-metal or RTOS-based systems
- *High-level programming focuses on algorithmic abstraction and portability, whereas embedded programming focuses on hardware control, timing constraints, and resource optimization.*

Embedded C

- **C (Standard C)**
 - Platform-independent (ANSI/ISO standards)
 - Executes under an operating system
 - No direct access to hardware registers
 - Memory and I/O handled via OS abstractions
 - Used for application and system software
- **Embedded C**
 - C language with hardware-specific extensions
 - Platform-dependent (microcontroller/processor specific)
 - Can run bare-metal or with RTOS
 - Direct hardware access using registers, ports, and interrupts
 - Supports special features: volatile, memory-mapped I/O, bit manipulation
 - Used for embedded system and real-time applications
- *Embedded C is not a different language, but a disciplined use of C tailored for hardware-oriented, real-time embedded*



- *Further Reading: List of different microprocessors (general purpose, application specific (embedded processors) and microcontrollers and their vendors with large market share*
- *Further Reading Major Microprocessors And Microcontrollers Lecture 1.pdf*